

AI Assisted Coding

Assignment 6.1

B.Tejaswi

2303A52123

Batch-40

Experiment 6: AI-Based Code Completion: Working with suggestions for classes, loops, conditionals

Task Description #1 (AI-Based Code Completion for Loops)

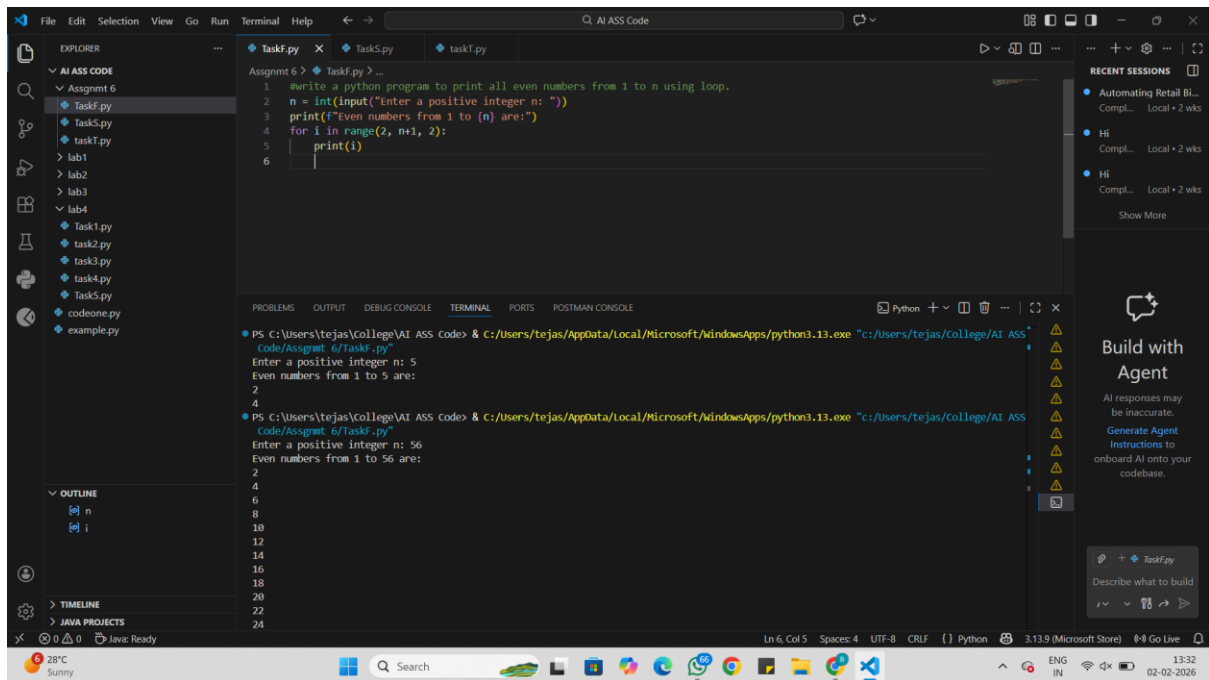
Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.



Analysis:

The program asks the user to enter a number **n**.

The number is stored in the variable **n**.

The loop starts from **2** because 2 is the first even number.

The loop increases by **2** each time.

Only even numbers are printed up to **n**.

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.

The screenshot shows a VS Code editor with a Python file named `task5.py`. The code is as follows:

```

1 #write a python program to count how many numbers in a list are even and odd.
2 numbers = [10, 21, 4, 45, 66, 93, 11]
3 even_count = 0
4 odd_count = 0
5 for num in numbers:
6     if num % 2 == 0:
7         even_count += 1
8     else:
9         odd_count += 1
10 print(f"Even numbers count: {even_count}")
11 print(f"Odd numbers count: {odd_count}")
12

```

The terminal output shows the execution results:

```

Code/Assgnmt_6/task5.py
Even numbers count: 3
Odd numbers count: 4
PS C:\Users\tejas\College\AI ASS CODE> & C:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\tejas\College\AI ASS

```

Analysis:

A list of numbers is stored in the variable **numbers**.

Two variables, **even count** and **odd count**, are set to 0.

The program checks each number in the list using a **for loop**.

If a number is divisible by 2, it is **even**, so even count is increased.

Otherwise, the number is **odd**, so odd count is increased.

Finally, the program prints the total count of even and odd numbers.

Task Description #3 (AI-Based Code Completion for Class Attributes Validation)

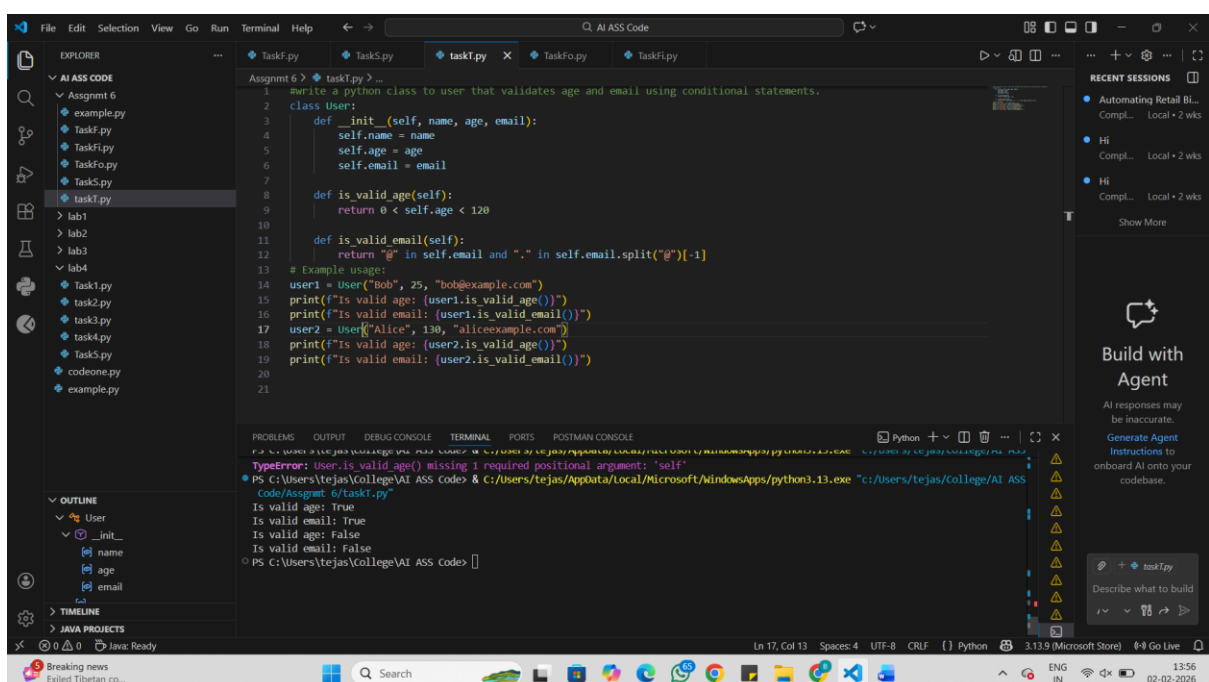
Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.



```
1 # write a python class to user that validates age and email using conditional statements.
2 class User:
3     def __init__(self, name, age, email):
4         self.name = name
5         self.age = age
6         self.email = email
7
8     def is_valid_age(self):
9         return 0 < self.age < 120
10
11     def is_valid_email(self):
12         return "@" in self.email and "." in self.email.split("@")[-1]
13
14 # Example usage:
15 user1 = User("Bob", 25, "bob@example.com")
16 print(f"Is valid age: {user1.is_valid_age()}")
17 print(f"Is valid email: {user1.is_valid_email()}")
18 user2 = User("Alice", 130, "alice@example.com")
19 print(f"Is valid age: {user2.is_valid_age()}")
20 print(f"Is valid email: {user2.is_valid_email()}")
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

TypeError: User.is_valid_age() missing 1 required positional argument: 'self'

PS C:\Users\tejas\College\AI ASS CODE> c:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:\Users\tejas\College\AI ASS CODE\Assignmt 6/task1.py

Is valid age: True
Is valid email: True
Is valid age: False
Is valid email: False

Analysis:

A class named **User** is created.

The **init** method stores the **name, age, and email** of the user.

The method **is valid age()** checks if the age is between **1 and 119**.

The method **is valid email()** checks if the email:

- contains **@**
- has a **.** after **@**

A user object is created and the validation results are printed.

The same checks are done for another user with invalid details.

Task Description #4 (AI-Based Code Completion for Classes)

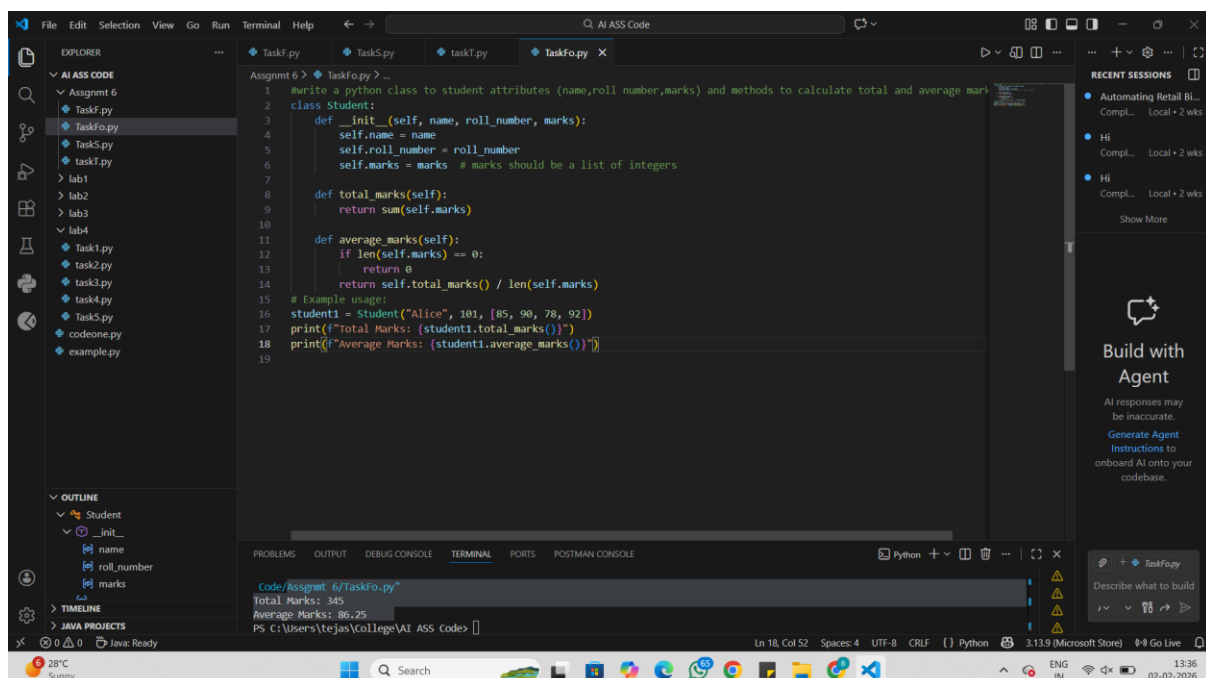
Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification.



```
1 write a python class to student attributes (name,roll number,marks) and methods to calculate total and average marks
2 class Student:
3     def __init__(self, name, roll_number, marks):
4         self.name = name
5         self.roll_number = roll_number
6         self.marks = marks # marks should be a list of integers
7
8     def total_marks(self):
9         return sum(self.marks)
10
11    def average_marks(self):
12        if len(self.marks) == 0:
13            return 0
14        return self.total_marks() / len(self.marks)
15
16    # Example usage:
17    student1 = Student("Alice", 101, [95, 90, 78, 92])
18    print(f"Total Marks: {student1.total_marks()}")
19    print(f"Average Marks: {student1.average_marks()}")
```

Code/Assignment_6/TaskFo.py
Total Marks: 345
Average Marks: 86.25
PS C:\Users\tejas\College\AI ASS CODE>

Analysis:

A class named **Student** is created.

The `__init__` method stores the student's **name, roll number, and marks**.

The **total marks()** method adds all the marks and returns the total.

The **average marks()** method:

- checks if the marks list is empty
- calculates the average by dividing total marks by number of subjects

A student object is created and the total and average marks are printed.

Task Description 5 (AI-Assisted Code Completion Review)

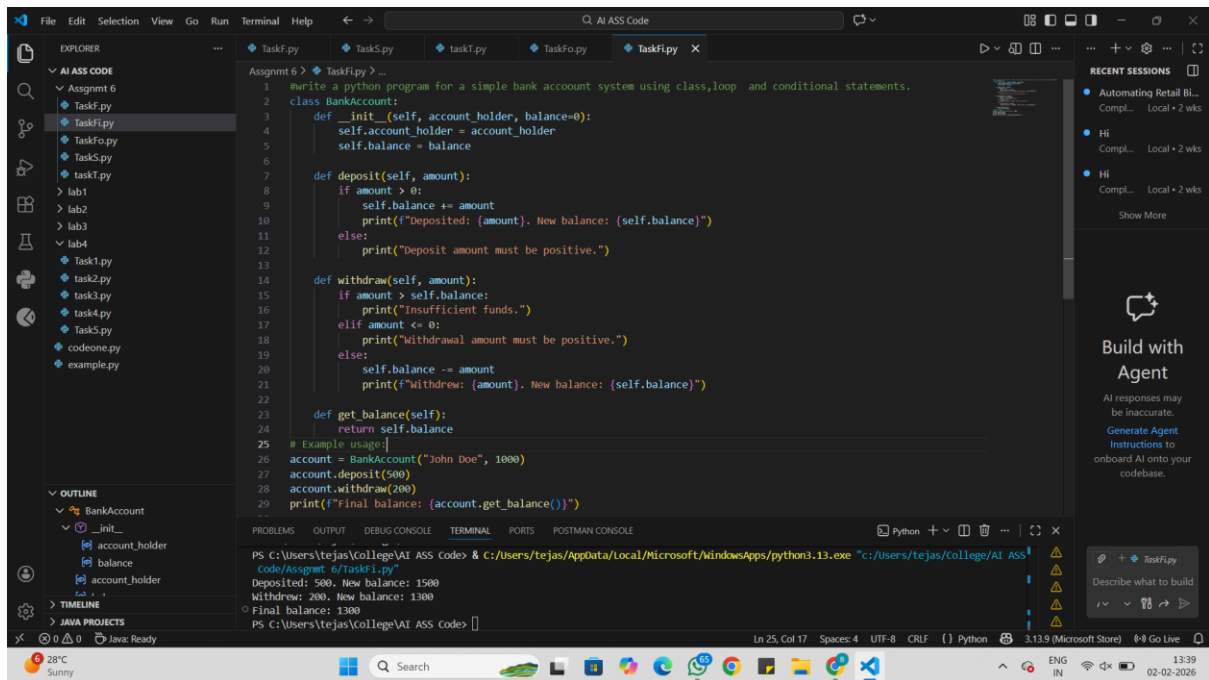
Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity.



Analysis:

A class named **BankAccount** is created.

The `init` method stores the **account holder name** and **balance**.

The **deposit()** method:

- checks if the amount is positive
- adds the amount to the balance

The **withdraw()** method:

- checks if there is enough balance
- checks if the amount is positive
- subtracts the amount from the balance

The **get_balance()** method returns the current balance.

The program performs deposit, withdrawal, and prints the final balance.